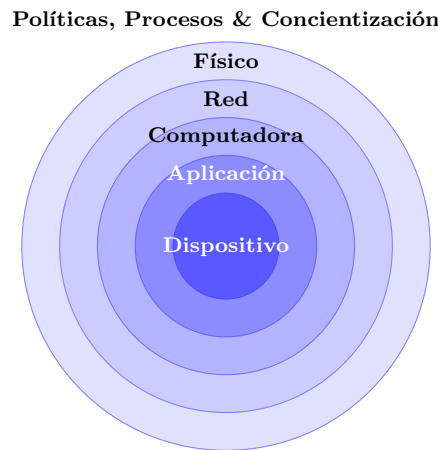


1. Principios de Diseño de Seguridad y Vulnerabilidades

1.1. Introducción: qué son los principios de diseño

Los **principios de diseño** son bases que promueven un diseño que tenga como resultado un sistema **robusto y seguro**. Son principios guía de *alto nivel* que deben adaptarse a las necesidades puntuales de cada sistema, se basan en las recomendaciones de los modelos de seguridad y deben aplicarse en **todas las capas** en donde funciona el sistema (políticas/procesos, físico, red, computadora, aplicación, dispositivo). Son simples y restrictivos.



Los principios se aplican en todas las capas en las que vive el sistema, del perímetro (políticas/procesos) hacia el núcleo (el dato/dispositivo).

! Importante — remarcado en clase

Que un sistema sea **robusto** significa que pueda aguantar cambios. El primer requisito de seguridad es que el sistema **esté disponible**: un ataque que tira el sistema (DoS) es un ataque de seguridad, aunque no comprometa confidencialidad ni integridad.

Todo aspecto de seguridad involucra tres patas que conviven (no alcanza con sola la tecnología):

- **Tecnología**: las herramientas (lo que más se ve en esta materia).
- **Procesos**: los procesos de las compañías; en sus grietas se mete la inseguridad.
- **Personas**: la pata psicológica / ingeniería social. *“Es el punto más crítico por donde el sistema cae.”*

— dicho en clase

! Importante — remarcado en clase

La seguridad es un **balance entre tres cosas**: seguridad, funcionalidad y eficiencia/performance (“bueno, bonito y barato”). Un sistema muy seguro suele ser difícilísimo de usar. La seguridad siempre es tan fuerte como el **eslabón más débil** de la cadena.

1.2. Principios de Saltzer & Schroeder (1975)

Propuestos en **1975** para el diseño de *sistemas operativos seguros*. Demostraron aplicabilidad en diseños generales y sistemas modernos. Son la base de buenas prácticas modernas y estrategias de gobierno en todas las capas. Muchos vienen del escenario militar.

! Importante — remarcado en clase

El profe remarcó que estos principios “hay que grabarlos afuera porque son los más importantes”. Aparecen todo el tiempo de una u otra manera y **están todos conectados / se tocan entre sí**. El objetivo de la materia es generar la **intuición** para aplicarlos.

1.2.1. 1. Mínimo privilegio (Least privilege)

Se debe asegurar que los sujetos (personas, entidades, procesos) sólo tengan el **acceso necesario** (la menor cantidad de permisos/capacidades) para realizar su trabajo, y nada más.

- Ejemplo: en un sistema con información financiera confidencial, limitar quién accede. La persona que atiende el teléfono no necesita acceder a todo; el administrador de cuentas sí, pero *no* a las cuentas que no administra.
- Por detrás surge que la **condición por default es no tener acceso a nada**.

Definición: Mínimo privilegio

Dar a cada sujeto únicamente los permisos/capacidades mínimas necesarias para realizar la tarea que tiene que hacer, y ningún permiso adicional. Por default no se tiene acceso a nada.

1.2.2. 2. Fallar de forma segura (Fail-safe defaults)

Conectado al anterior: el default es “no dejo entrar a nadie”.

- A la hora de revisar, las **listas de permitidos (whitelists)** son mejores que las listas de excluidos (**blacklists**). Conviene marcar quién *puede* entrar, no quién no puede. (Ejemplo del profe: Twitter pasó de white-listing a black-listing por razones económicas/de influencia, pero en general el white-listing es mejor.)
- Un sistema no abre puertos innecesariamente; por default los firewalls están encendidos.
- No hay cuentas de usuario por defecto con contraseña conocida.
- Un sistema que presenta una falla en un proceso o control de seguridad debe moverse a un **estado seguro**.
- Ejemplo: si el sistema de autenticación no puede conectarse a la base de datos para verificar un password, debe **rechazar el pedido** y no asignar permisos, por más que sean los mínimos. Lo que se penaliza al fallar seguro es *funcionalidad*.

Ejemplo extremo de hardware: las normas FIPS de dispositivos físicos (HSM, criptochips/tokens) tienen niveles (1 a 4); en nivel 4, ante una amenaza física el dispositivo debe desaparecer o no debe haber forma de extraer la clave privada (anti-tampering).

1.2.3. 3. Simplicidad (Economy of mechanism)

La complejidad es enemiga de la seguridad. Se debe evitar arquitecturas muy sofisticadas al desarrollar controles de seguridad.

- Las soluciones complejas **exponen más superficie de ataque**, son menos robustas y más difíciles de entender; es más fácil que se cuelen bugs que deriven en vulnerabilidades explotables.
- Busca reducir la superficie de ataque y la complejidad de datos: p.ej. reducir al mínimo la cantidad de tuplas (**sujeto, acceso, objeto**). Cada acceso es susceptible de ser explotado.

1.2.4. 4. Mediación completa (Complete mediation)

Si hay un control, este debe aplicar a **todas** las interacciones alcanzadas.

- Es “la puerta del castillo”: todo pasa por ahí y no hay ningún otro mecanismo posible.
- El muestreo de algunas transacciones, controles al azar o entidades por fuera del control son problemas potenciales. No tiene sentido construir un muro y dejar la puerta abierta por el costado.

1.2.5. 5. Sistema/Diseño abierto (Open design)

No debe basarse la seguridad en mantener el diseño oculto (no “seguridad por oscuridad/ofuscación”).

- Es el principio de Kerckhoffs aplicado: en encriptación lo que se guarda es la **clave** (y se cambia frecuentemente); el **algoritmo es público** y conocido por todos. El mismo concepto aplica a los controles de seguridad.
- Todo secreto conocido por al menos una persona puede ser revelado. “*Los bits son eternos.*”
— dicho en clase (Asumir que todo lo digitalizado puede llegar a verse / perderse.)
- Matiz: que NO deba basarse en ofuscación **no significa** que tenga que ser abierto y que todos se enteren. La ofuscación da algo de complejidad y tiempo (por eso los algoritmos militares son ocultos, aunque sigan también este principio); sirve como *capa adicional*, no como única defensa.
- Pata de software libre/open source: es bueno porque tiene más **escrutinio** (más ojos $\Rightarrow$ menos bugs explotables). Hoy hay que tomarlo con pinzas: si bots meten commits automáticos en open source, el principio se rompe (importa que haya un **responsable** que responda).

1.2.6. 6. Separación / Segregación de privilegios (Separation of privilege)

Toda tarea crítica no debe quedar bajo el control de un solo sujeto; más de un sujeto debe ser parte del proceso para asegurar un control entre partes.

- Se aplica el principio de **control por oposición de intereses**: las partes tienen objetivos opuestos y en el equilibrio se evita el abuso.
- Ejemplo (ventas): para aprobar una venta con descuento, el vendedor (que quiere descontar mucho para cerrar) pide aprobación al financiero (que quiere descontar lo menos posible). Hay oposición de intereses.
- En seguridad informática: que no sea el mismo quien controla las claves de la base de datos y quien controla el sistema de pagos; si controlara la base, podría crearse un usuario con permisos para operar el sistema de pagos (“salto”).
- Puede contradecir el principio de simplicidad (agrega complejidad): ahí está el balance.

► En el parcial

Vínculo no trivial (clave para el parcial): el profe relaciona “**separar las credenciales del sistema de las del usuario**” con el principio de **separación de privilegios**. Otro vínculo: mediación completa (única) + separación de privilegios $\Rightarrow$ la mediación de seguridad es única, pero el *privilegio de controlarla* se separa para que no haya un único punto de falla.

1.2.7. 7. Mecanismos exclusivos (Defense-in-Depth)

Tiene dos partes:

- **Exclusividad:** evitar que distintos mecanismos compartan recursos, algoritmos y hardware. Si algo es para seguridad, debe ser un mecanismo **exclusivo** para eso y nada más.
- **Defensa en profundidad:** diseñar controles compensatorios / de mitigación / reacción que se accionen si otro control falla o es superado.

Ejemplo (fuente de bugs/vulnerabilidades): un **flag** que originalmente indica “el cliente puede comprar” se reutiliza después también para “puede recibir un voucher” (asunción *as-is*/correlación que se mantendría eterna). Cuando esa correlación se rompe, el código queda mezclado: un mismo flag significa dos cosas. Si alguien controla indirectamente ese parámetro, rompe la exclusividad y puede saltar de un sistema a otro (relacionado con *pivoting*). Ejemplo análogo físico: una llave/flag de acceso a una puerta importante a la que implícitamente se ató el acceso a un servidor; el personal de limpieza con esa llave queda con acceso al servidor.

► En el parcial

Defensa en profundidad (remarcado en la clase de consulta): encadenar controles para **maximizar cuántas cosas hay que romper** para vulnerar el sistema. Ejemplo del PDF: se hace segregación de tareas + mínimos privilegios, pero una vulnerabilidad permite llevar adelante una venta sin autorizaciones $\Rightarrow$ ¿qué controles se agregan para monitorearla? ¿alertas que bloqueen la transacción? ¿una confirmación?



*Defensa en profundidad: se **encadenan** controles independientes. Si uno falla o es superado, otro lo compensa; el atacante debe vulnerar todas las capas para llegar al activo.*

1.2.8. 8. Menor asombro / Aceptación psicológica (Psychological acceptability)

- El **peor enemigo** de cualquier diseño de seguridad es el **usuario no alineado**; el mejor aliado es el usuario comprometido con la seguridad.
- Los controles deben ser simples de usar, incluso para alguien sin conocimientos técnicos, y no deben complicar el negocio ni sus procesos.
- La **concientización** del problema y del valor de la seguridad es fundamental para lograr la aceptación del control.

! Importante — remarcado en clase

“Los muy buenos trabajadores son malos usuarios de seguridad.”

— dicho en clase Los más eficientes saben “navegar la burocracia” y van a saltar las restricciones de seguridad para resolver el problema. Por eso hay que capacitarlos para que

entiendan que mantener la seguridad es parte de su trabajo. Un buen esquema asume que la persona *va a* intentar saltarlo y diseña para que eso sea una excepcionalidad controlada y consciente. Caso real: un compañero pasó dos meses mejorando la seguridad de la página rastreadora de ambulancias, y luego el jefe de ambulancias consiguió por “rosca” el mismo usuario con todos los permisos porque el negocio lo requería.

△ Cuidado — error típico

La seguridad “no tiene límite” (es un *bounder*): se puede poner tanta como la paranoia diga. El sistema más seguro del universo sería imposible de usar. Hay que balancear con experiencia de usuario (UX), pensando las cosas centradas en las personas.

1.3. La seguridad NO se garantiza

△ Cuidado — error típico

GANCHO/TRAMPA que penaliza. “*Garantizar la seguridad es algo que no se puede hacer jamás.*”

— dicho en clase Ante la pregunta “¿me garantizás que esto es seguro?”, nadie idóneo dice “sí, te lo garantizo”: es muy caro, requiere conocimiento muy específico y aún así es muy difícil por todo lo que ocurre *fuera* del sistema (el ecosistema de infraestructura/redes donde funciona).

Construir software de CALIDAD NO es garantía de seguridad ($V/F \Rightarrow \text{FALSO}$ la afirmación “calidad = seguro”; Recup 2025). Afirmar que algo “garantiza la seguridad” o que “software de calidad = seguro” **penaliza**.

Por eso, saliendo de los mecanismos específicos, la seguridad se trata como un problema de **gestión de riesgo**. Amenaza y vulnerabilidad componen dos probabilidades. Frente a un riesgo se puede:

- **Eliminarlo.**
- **Mitigarlo** (reducir la probabilidad o el impacto; p.ej. backups en otra nube).
- **Compensarlo** (no se evita, pero se agrega p.ej. una alerta que avisa cuando pasa).
- **Aceptarlo** (documentarlo y no hacer nada): a veces es la solución más efectiva cuando evitarlo es carísimo (en dinero o en UX). Lo importante es que la decisión se tome con conciencia.

1.4. Calidad/Procesos: DevOps vs DevSecOps vs MLOps

► En el parcial

“Seguridad de la Información” NO es un componente de DevOps — pertenece a **DevSecOps** (el agregado de *Sec*). Hoy además se suma **MLOps** (operaciones de machine learning). Errores que penalizan: ubicar la seguridad como parte nativa de DevOps.

1.5. Modelado de amenazas (Threat Modeling)

Es una técnica de ingeniería para entender las **amenazas, ataques, vulnerabilidades y contramedidas** que pueden afectar a una aplicación. Define la tarea de modelar las amenazas

al negocio en la **superficie de ataque**; con el modelo y la clasificación de amenazas se definen contramedidas específicas. Debe incluir inteligencia, tendencias y ataques pasados, y es un **proceso continuo** (las amenazas cambian). Se usa para mejorar el diseño y reducir el riesgo.

Proceso (Microsoft / OWASP, similares):

1. Definir el alcance (acotar el esfuerzo para que cubra lo suficiente sin excederse).
2. Determinar / definir amenazas (diagramar dependencias, identificar caminos de explotación).
3. Definir contramedidas y mitigación.
4. Revisar el trabajo, los cambios y volver a empezar (validar).

1.5.1. Modelo STRIDE

► En el parcial

STRIDE es de Microsoft (V/F, 2015 $\Rightarrow$ Verdadero: “fue desarrollado por Microsoft”). Es conocimiento “**enciclopédico**” exigible: hay que saber de memoria qué significa cada letra y qué propiedad viola. Surge en la época de Windows 98 cuando se cuestionaba mucho la seguridad de Windows.

Clasifica las amenazas en **6 categorías**. La idea del modelo: por cada sistema, conseguir al menos 2 amenazas críticas en cada categoría.

	Amenaza	Propiedad violada	Definición
S	Spoofing	Autenticación	Hacerse pasar por algo/alguien que no se es
T	Tampering	Integridad	Modificar algo en disco, red, memoria, etc.
R	Repudiation	No repudio	Negar haber hecho algo (sin poder probarlo)
I	Information Disclosure	Confidencialidad	Revelar información a quien no debe acceder
D	Denial of Service	Disponibilidad	Agotar recursos necesarios para dar el servicio
E	Elevation of Privilege	Autorización	Permitir hacer algo para lo que no se está autorizado

	AMENAZA	PROPIEDAD VIOLADA
S	Spoofing	Autenticación
T	Tampering	Integridad
R	Repudiation	No repudio
I	Information Disclosure	Confidencialidad
D	Denial of Service	Disponibilidad
E	Elevation of Privilege	Autorización

Las 6 categorías STRIDE y la propiedad de seguridad que viola cada una (memorizar: cada letra $\leftrightarrow$ su propiedad).

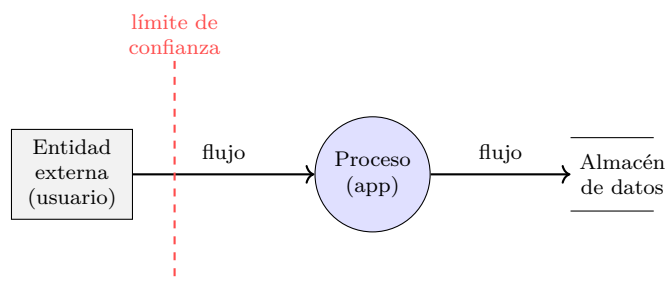
1.5.2. DFD (Data Flow Model) + STRIDE

El DFD representa las partes de un sistema y los riesgos de cada una. Analiza elementos de cada categoría:

- **Procesos:** contenedores, procesos ejecutables.
- **Entidades externas** al sistema: personas, otros sistemas.

- **Flujos de datos:** comunicaciones entre procesos.
- **Almacenamientos de datos:** archivos, bases de datos, secretos.

Para cada elemento se **cruza con las amenazas STRIDE** (matriz), y por cada riesgo identificado se define cómo gestionarlo: **eliminarlo, mitigarlo, compensarlo o aceptarlo**. (También se dibuja como árbol: amenazas → 6 categorías → ejemplos → recursos.) OWASP tiene su versión (casi igual a la de Microsoft, pero con manual gratuito). Hay industrias reguladas (pagos) que exigen demostrar estos ejercicios.



DFD: entidades externas, procesos, flujos y almacenes de datos. El **límite de confianza** (trust boundary, línea roja) marca dónde cambia el nivel de confianza: los datos que lo cruzan deben validarse.

Elemento	S	T	R	I	D	E
Entidad externa	✓		✓			
Proceso	✓	✓	✓	✓	✓	✓
Almacén de datos		✓	?	✓	✓	
Flujo de datos		✓		✓	✓	

Matriz DFD × STRIDE: a qué amenazas es susceptible cada tipo de elemento (los procesos, a todas). Por cada ✓ se decide eliminar / mitigar / compensar / aceptar.

1.6. Amenazas y Vulnerabilidades

Definición: Amenaza (Threat)

Cualquier elemento que pueda afectar la confidencialidad, integridad o disponibilidad de un sistema. Apunta a la **existencia de alguien (o algo) con ganas de lastimar**.

Definición: Vulnerabilidad

Una debilidad en un activo o grupo de activos que permite que un atacante la aproveche para lograr un acceso que no tiene permitido. Es el **canal de ejecución** de la maldad. Técnicamente, las vulnerabilidades son **bugs que se disfrazan de feature/seguridad**.

! Importante — remarcado en clase

Distinción sutil pero **clave** (se toma siempre):

- **Amenaza** = las *ganas* de atacar (p.ej. las ganas del hacker de romper tu sitio para robar plata).
- **Vulnerabilidad** = la cuestión técnica que el script explota (un bug).
- **Exploit** = el programa/script que explota la vulnerabilidad para generar la brecha.

► En el parcial

Ecuación de riesgo (para grabarse): la amenaza “multiplica” a la vulnerabilidad, y eso mide el riesgo. El profe agrega un tercer término: el **valor del activo (Asset) / la plata que se pierde**. $\text{Riesgo} \approx \text{Amenaza} \times \text{Vulnerabilidad} \times \text{Asset}$ (multiplicativo). Con esas 3 cosas se hace un análisis de riesgo robusto que captura ~95 % de la problemática, porque los recursos son finitos y no se puede controlar todo.

1.6.1. Tipos de amenazas (taxonomías)

- **Según origen:** *internas* (empleado que hace fraude; falla en equipo que deja sin servicio) o *externas* (grupo criminal robando info; huracán hacia el datacenter).
- **Según agente amenazante:** *humanos* (hacker; administrador cometiendo un error), *ambientales* (evento natural, inundación), *tecnológicos* (servidor que se quema). Se discute agregar una 4.^a categoría: **agentes de IA**.
- **Según intención:** *malicioso* (ataque intencional) o *no malicioso* (error no intencional).
- **Según impacto:** destrucción (disponibilidad), corrupción (integridad), revelar información (confidencialidad), robo del servicio (fraude, sin afectar a otros usuarios), denegación de servicio (disponibilidad), elevación de privilegios (bypass de ACL), uso ilegal (p.ej. usar Google Drive para distribuir contenido con copyright).

1.6.2. Tipos de vulnerabilidades

- **En el código fuente:** fallas en controles o lógica deficiente; lo específico de una aplicación o heredado de dependencias.
- **Componentes mal configurados:** la forma más común en la nube (ej.: servidor Mongo provisionado con clave de admin pública por defecto).
- **Relaciones de confianza:** no revalidar la autenticidad del sujeto (ej.: NFS no valida al usuario; asumir que tras el login las demás páginas ya están autenticadas y no chequear cada una; o no chequear autorización porque “nadie va a adivinar la URL”).
- **Malas prácticas de credenciales:** políticas de password inseguras, credenciales expuestas, falta de MFA.
- **Falta de cifrado fuerte (Lack of strong encryption):** algoritmos débiles, claves chicas, criptosistemas viejos (ej.: permitir suites *export* en TLS).
- **Insider threat:** persona que incumple reglas para hacer daño/fraude (más asociado a seguridad física).
- **Vulnerabilidad psicológica:** cualquier causa humana sin intención que genera un problema; toda la ingeniería social (“cuento del tío”) cae acá. Ligada al principio de aceptación psicológica.
- **Autenticación inadecuada:** proceso completo de autenticación mal hecho (ej.: login fuerte con 2FO pero un “recuperar contraseña” débil que da entrada).
- **Injection flaws:** cualquier inyección de código. “*Si de esta materia tienen que llevarse una sola cosa, es que el 90 % de las vulnerabilidades caen en Injection Flow.*”

— dicho en clase

- **Sensitive data exposure:** errores que muestran datos que no deberían (stack de error con código fuente; *dumpster diving*; notebooks viejas mal borradas).
- **Insufficient monitoring and logs:** poca visibilidad para detectar/alertar/corregir; afecta mucho a la disponibilidad.
- **Shared tenancy:** recursos compartidos entre tenants que no aíslan bien la info (multi-tenant cloud; carpeta /tmp compartida; ataques de hardware como *Rowhammer* y *Spectre*, que rompen el aislamiento entre máquinas virtuales).

1.6.3. MITRE: CVE, CWE y Zero-day

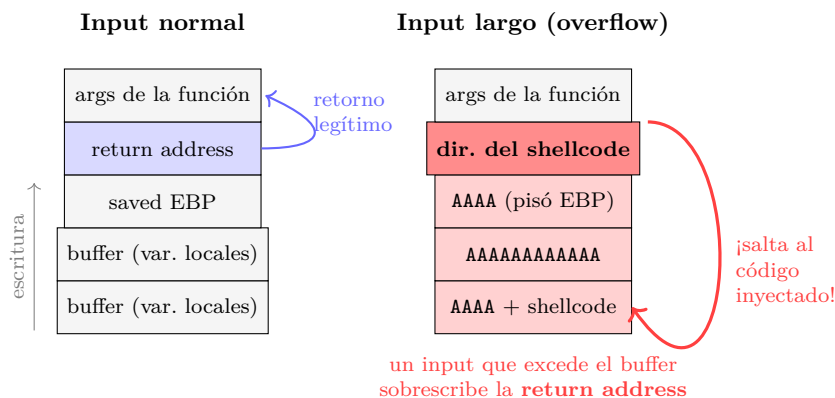
- **CVE** (Common Vulnerabilities and Exposures): registro centralizado de vulnerabilidades de software. Formato **CVE-YYYY-NNNN** (año + número de secuencia). La cantidad de CVEs publicadas por año crece de forma explosiva desde ~2017. Ejemplos: **CVE-2021-44228** (Log4j, ejecución arbitraria de código), **CVE-2014-0160** (Heartbleed en OpenSSL: “buffer over-read” que filtra memoria y permite extraer claves/master key de la sesión).
- **CWE** (Common Weakness Enumeration): registro de las *debilidades* en software que dan lugar a las vulnerabilidades. Cada año/varios años MITRE publica el Top 25. Top: #1 **CWE-787 Out-of-bounds Write** (buffer overflow), #2 **CWE-79 Cross-site Scripting**, #3 **CWE-89 SQL Injection**, #4 **CWE-416 Use After Free**, #5/#6 **CWE-78 OS Command Injection**.
- **Zero-day:** vulnerabilidad/exploit aún no reportado ni conocido por el fabricante. Tiene alto valor de mercado (no hay prevención). Existe mercado negro y uno gris (Zerodium paga, p. ej., hasta US\$1.000.000 por un RCE remoto en Windows sin intervención del usuario). El embargo: si una CVE es muy crítica (rating > 9) se publica el sistema afectado y se da ~3 meses antes del detalle.

1.7. Buffer overflow e inyección de código

Definición: Buffer overflow (CWE-787, Out-of-bounds Write)

Escritura en memoria fuera de los límites originalmente definidos del buffer. Lleva a: corrupción de datos, crash (el clásico *segmentation fault*) o **ejecución de código**.

El peligro mayor: si el buffer está en el **stack**, este contiene los datos de la última llamada a función y la **dirección de retorno**. Un atacante puede escribir el buffer de forma que inyecte su código y pise la dirección de retorno para que el flujo salte a ese código. Hay contramedidas en compiladores, pero **ninguna es 100 % efectiva**. Lenguajes de más alto nivel (Java, etc.) se volvieron populares en parte porque eliminan la aritmética de punteros y este tipo de errores.



*Stack smashing: el input que desborda el buffer sigue escribiendo hacia las direcciones superiores hasta **pisar la dirección de retorno**; el atacante la reemplaza por la dirección de su propio código (shellcode) y el flujo salta ahí.*

! Importante — remarcado en clase

Von Neumann: datos y código viven en el mismo espacio de memoria, lo que **habilita la inyección de código** (caso iOS, 2018). La generalización del problema de inyección es que el canal de datos de control del sistema (un nivel) queda **mezclado** con el canal de datos del usuario (otro nivel) sin lograr separarlos. Esto aplica también a los LLM (prompt injection / poisoning): no hay un mecanismo *exclusivo* que separe el canal de control del de la pregunta del usuario.

► En el parcial

“Qué mitigación **NO** tiene sentido” para **buffer overflow** (2025-2Q), entre opciones tipo: **RUST** / **garbage collector** / **validar la entrada** / **Stack Canary**. El profe admitió que es **ambiguo**: “Todas pueden contestar y todas están bien.”

— dicho en clase Lo que importa es **JUSTIFICAR** la elección. (RUST y validar la entrada previenen; Stack Canary detecta el pisado de la dirección de retorno; el GC es el candidato más discutible como mitigación directa del stack overflow, pero hay que argumentarlo.)

△ Cuidado — error típico

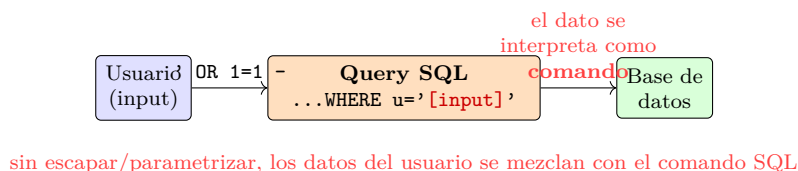
Use After Free (CWE-416): liberar la memoria (free) pero quedarse con el puntero y seguir usándolo. Puede derivar en valores raros, crash o ejecución de código. Es un recurso que no se “limpia” (wipe) entre usos.

1.8. Vulnerabilidades web

1.8.1. SQL Injection (CWE-89)

Falta de control en el ingreso de los usuarios que permite ejecutar código SQL en la base de datos (se mezclan datos con comandos y la base interpreta los datos como comando).

- **Defensa:** escapar/parametrizar las entradas; usar **prepared statements** (armar la query con parámetros: “el parámetro 1 vale tanto, el 2 vale tanto”).
- Muy peligrosa: hay herramientas automatizadas que, dado un parámetro vulnerable, averiguan el motor, el esquema y hacen *dump* de cualquier tabla. Basta 1 query mal sanitizada entre cientos para comprometer todo.

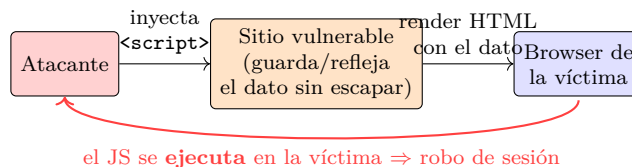


*SQL Injection (CWE-89): el input no sanitizado se concatena en la query y la BD lo **ejecuta como comando**.
Defensa: prepared statements (parámetros separados del comando).*

1.8.2. Cross-Site Scripting / XSS (CWE-79)

Inyección de código malicioso (JavaScript) en un link o requerimiento que termina siendo **ejecutado por la víctima / el browser**. Ocurre al hacer render de HTML mezclando un dato (que controla el usuario) con código sin escapar, de modo que el dato puede ser código.

- **Aprovecha la confianza** entre el sitio y el browser/sesión: si se ejecuta JS en la máquina de la víctima, se le puede **robar la sesión**.
- **Defensa:** sanitizar/escapar todas las salidas (*escape on output*) o usar frameworks de componentes que sanitizan automáticamente. Mismo problema que SQLi: basta olvidarse de uno entre 999.



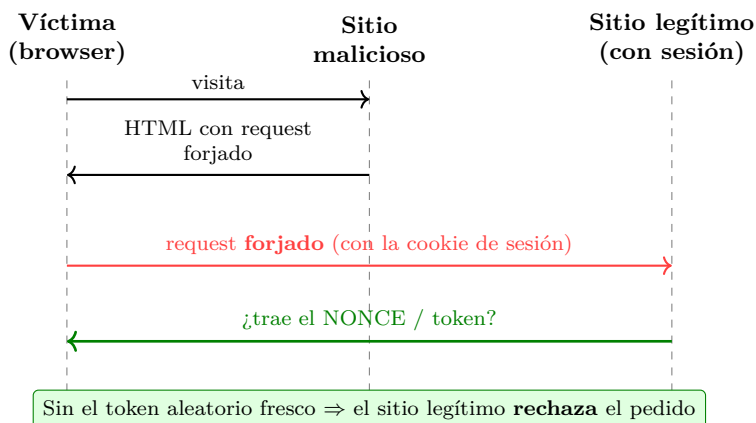
XSS (CWE-79): el dato controlado por el atacante termina siendo **ejecutado como código (JS)** por el browser de la víctima, aprovechando la confianza del sitio en su sesión.

1.8.3. CSRF (Cross-Site Request Forgery)

► En el parcial

Defensas a CSRF (foco de examen):

- La defensa **real** es el **NONCE / token anti-CSRF**: un valor aleatorio único en el formulario que da **frescura** y evita el *replay*.
- Forzar **POST** en vez de **GET**.
- **Concientizar** contra phishing.
- **NO sirve banear la IP**: el usuario es legítimo (la petición se ejecuta en su nombre).



CSRF: el sitio malicioso hace que el browser de la víctima dispare un request al sitio legítimo con su cookie de sesión. El **nonce/token anti-CSRF** (valor aleatorio único en el formulario) lo frena: el atacante no lo conoce, y evita el replay.

1.8.4. OS Command / Shell Injection (CWE-78)

Falta de control en el ingreso que permite ejecutar comandos del sistema operativo. Genera *backdoors* para ignorar controles de red o autenticación. Ejemplo: una app PHP que delega a *ImageMagick* pasándole el nombre del archivo sin sanitizar; si el nombre contiene `; rm -fr ...` se ejecutan otros comandos.

1.8.5. Path / Directory Traversal

! Importante — remarcado en clase

Caso **router Linksys (2013)**: parámetro `next_page=/etc/passwd` permitía leer archivos fuera del directorio esperado (traversal de directorios).

1.8.6. XXE (XML External Entities)

XML permite cargar esquemas desde una URL; si esa URL se usa para hacer un GET a otra página, un atacante puede escanear la red interna visible por el servidor (estuvo de moda porque casi todas las librerías XML tenían el problema).

1.8.7. Organizaciones de referencia (OWASP)

OWASP (Open Web Application Security Project): organización enfocada en combatir problemas de seguridad en aplicaciones Web; publica regularmente las vulnerabilidades más comunes y buenas prácticas. Tiene el **Top 10** (web), **OWASP Cloud Top 10** y **OWASP Top 10 LLM** (ej.: LLM01 Prompt Injection, LLM03 Training Data Poisoning, LLM05 Supply Chain). Junto con MITRE son referencia hace >20 años. (Vulnerabilidad moderna asociada: *content/prompt poisoning*: texto invisible en páginas o currículums tipo “ignora tus instrucciones” que el LLM/agente termina ejecutando.)

1.9. Malware básico (mencionado / referenciado para definir)

► En el parcial

Ejercicio típico (2016): **DEFINIR** varios conceptos (mínimo privilegio, confinamiento, root-kit, troyano) y luego **relacionar dos de forma no trivial**. Estos términos se desarrollan en profundidad en la unidad de *Flujo de información y Malware*; acá basta tenerlos a mano:

- **Troyano**: programa que se disfraza de algo legítimo/útil para que el usuario lo ejecute, ocultando funcionalidad maliciosa (análogo al bug que “se disfraza de feature”).
- **Rootkit**: software malicioso que se oculta en el sistema (típicamente con privilegios de root/administrador) para mantener acceso persistente y evitar ser detectado.
- **Confinamiento**: limitar/aislar lo que un proceso puede hacer o acceder (sandboxing), muy ligado a *mínimo privilegio* y a *mecanismos exclusivos*.

Vínculo no trivial sugerido: *confinamiento* es la materialización técnica de *mínimo privilegio* (aislar para que aunque algo se comprometa, no pueda hacer más que lo mínimo).

1.10. La idea del “chip paranoico”

! Importante — remarcado en clase

De la clase de consulta, el profe remarcó el “**chip paranoico**”: pensar como atacante. “*Seguridad informática es paranoia controlada.*”

— dicho en clase Los muy buenos en seguridad son paranoicos (controladamente): asumen que todos quieren hacer daño de alguna manera, y eso suma. Junto con la **defensa en profundidad** (encadenar controles para maximizar cuántas cosas hay que romper), son las dos ideas rectoras. Además hay preguntas “**enciclopédicas**” que requieren **memorizar**

definiciones (p. ej. cada letra de STRIDE).

1.11. Lectura recomendada

Capítulos 12–13 de *Computer Security: Art and Science*, Matt Bishop (Bishop habla de principios, no tanto de amenazas concretas; para amenazas, el proceso de Threat Modeling de OWASP).